Warranty Vault



SE4041 Assignment 02 – part A

Project Report

Student name : Hitiarachchi A M L

Student ID – IT22079640

1. Introduction and purpose

Warranty Vault is an iOS application designed to help people keep purchase receipts and track product warranty periods in one place. Paper receipts can be misplaced, and users may forget when a warranty expires. This can make it harder to request repairs or warranty support for phones, laptops, appliances, and other valuable purchases. The application provides a digital record of each purchase and schedules reminders before the warranty ends.

The main workflow allows a user to import a receipt image or PDF from Photos or Files. The app uses on-device text recognition to identify likely values such as store, purchase date, total amount, and category. These values are presented for review before the user saves the warranty item. Uncertain fields are highlighted in amber, and the recognized text can be expanded for checking. The user selects a warranty length of 6, 12, 24, 36 months, or a custom period. The app then calculates the expiry date and schedules reminders 30, 7, and 1 day before expiry.



FIGURE 1: app icon

2. Target audience and justification

The intended audience is students and young families in Sri Lanka who purchase high-value electronics and household products. These users may keep receipts in bags, drawers, email attachments, or photo galleries, making it difficult to retrieve them when needed. A searchable digital vault reduces reliance on paper and gives users a clear view of active, soon-to-expire, and expired warranties.

The design prioritises a simple flow: import or enter purchase details, check uncertain values, choose the warranty period, and save. Search, filters, list/grid views, insights, and reminders support day-to-day use without requiring users to understand technical details.

3. Requirements mapping

Requirement / user need	Implementation	Where to find it
Store purchase and warranty information	WarrantyItem model persisted with SwiftData	Add flow; vault; item detail
Read receipt text	Vision text recognition in accurate mode	Import and review flow
Help users verify OCR	Amber confidence indicators and collapsible raw text	Review form
Calculate warranty expiry	Computed expiryDate and status	Item detail; vault cards
Warn before expiry	Local reminders scheduled 30, 7, and 1 days before expiry	Notifications; settings
Browse saved items	List/grid, search, filter chips, swipe actions, context menu	Vault
Summarise purchases	Charts for category, monthly spending, and upcoming expiry	Insights
Support export and preferences	CSV export, appearance and reminder settings	Settings
Support English and Sinhala interface text	String Catalogs	App interface

4. Design decisions and trade-offs

4.1 On-device receipt recognition

The app uses Apple Vision to recognise receipt text on the device. This avoids sending receipt images to an external recognition service and supports a privacy-focused workflow. OCR is not guaranteed to be correct, especially with faded, crumpled, or poorly lit receipts. For this reason, the app provides confidence cues and shows the raw recognised text so users can review and correct values.

4.2 Importing files rather than live camera capture

Receipt images can be selected through PhotosPicker, and PDFs or images can be imported through the file importer. Live camera capture is not included in this version because the development environment could not target the available physical iPhone and the simulator had no camera. File import still supports the core receipt-entry task.

4.3 SwiftUI and MVVM

SwiftUI provides a declarative interface, while MVVM separates screen presentation from business logic. This makes the code easier to test and maintain. A repository isolates SwiftData access, and protocols allow services such as text recognition and reminder scheduling to be replaced with test doubles.

4.4 Confidence-aware review

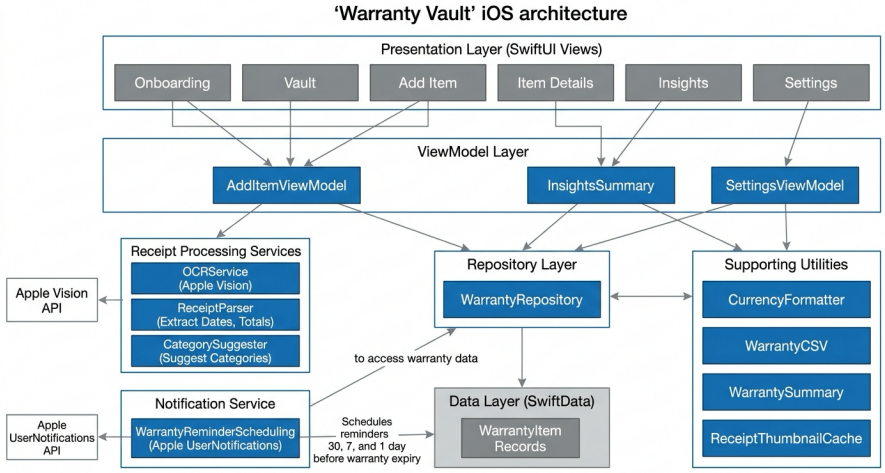
Automatically recognised values are suggestions, not unquestionable facts. Amber highlights and a collapsible text section help users identify possible OCR mistakes before saving. This adds a review step but reduces the risk of saving incorrect purchase or expiry information.

5. Architecture

Warranty Vault follows MVVM with a repository pattern. Views display data and forward user actions to view models. The WarrantyRepository is the only type that directly interacts with ModelContext. Services handle OCR, receipt parsing, category suggestions, image/PDF handling, and reminder scheduling.

The central WarrantyItem model stores purchase and warranty details, with computed expiry date and status. AppRouter manages a typed navigation path for each tab. Protocols make important services replaceable in tests, including a mock reminder scheduler.

High-level flow: User interface → ViewModel → Repository / service layer → SwiftData or iOS frameworks.



6. Implementation of key features

6.1 Receipt import and OCR

The user imports an image or PDF. Image decoding and PDF rendering are performed away from the main thread, and PDF rendering is capped to control memory use. Vision's VNRecognizeTextRequest runs in accurate mode with language correction. The recognised text is passed to ReceiptParser, which extracts candidate fields and confidence information.

6.2 Date and amount parsing

Date parsing first looks for labels such as invoice date, bill date, transaction date, purchased, date, or dt. It can also inspect a label on the previous line before falling back to other date patterns. Supported formats include day-first numeric dates, ISO-style dates, month names, dates with spaces or attached times, and common OCR character confusions. Future dates and dates more than 15 years old are rejected, while ambiguous matches receive low confidence.

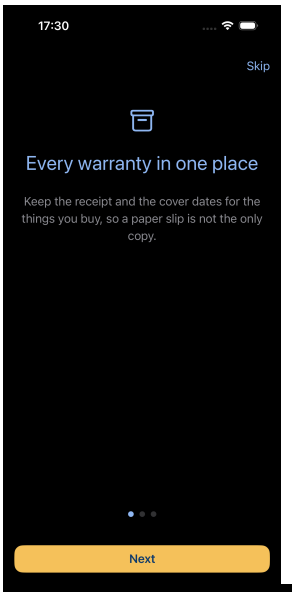
Amount parsing ranks labels such as grand total, net total, amount due, and total. Cash, change, tendered amount, VAT, tax, discount, subtotal, and quantity lines are excluded as total candidates. A previous test revealed that a change value could be selected instead of the receipt total; the parsing rules were updated to prioritise the correct total and use cash-minus-change as supporting evidence when it matches.

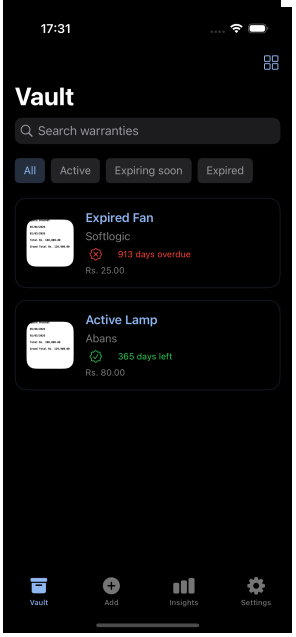
6.3 Persistence and navigation

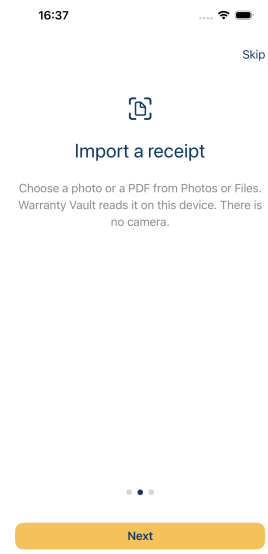
SwiftData stores warranty items. WarrantyRepository centralises persistence operations, while ModelContainerFactory configures the data container. AppRouter uses a typed Route enum and a NavigationPath for each tab. The vault supports list and grid layouts, search, filters, swipe actions, and a context menu.

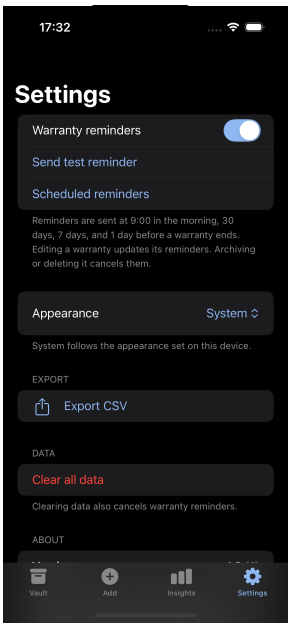
6.4 Reminders and insights

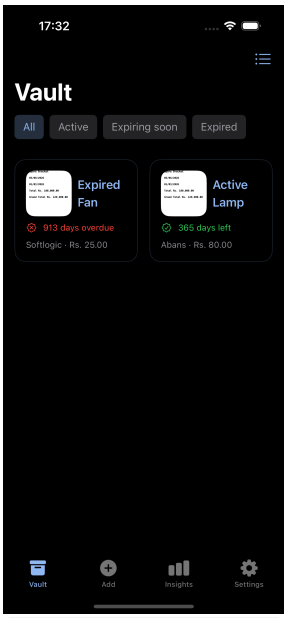
The UserNotifications implementation schedules local notifications for 30, 7, and 1 day before expiry. Settings include a reminder toggle, a five-second test reminder, and a scheduled-reminders list. The Insights screen uses Swift Charts to present category distribution, monthly spending, upcoming expiries, and a summary.

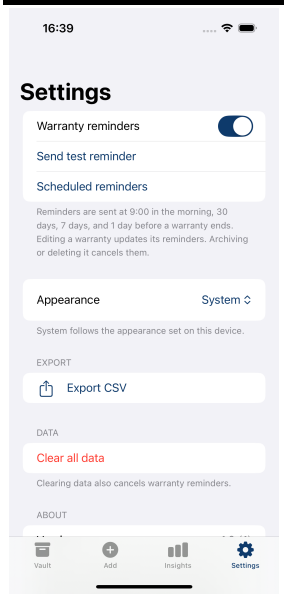


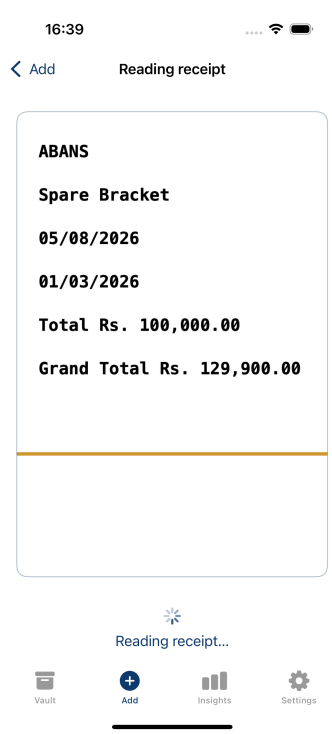


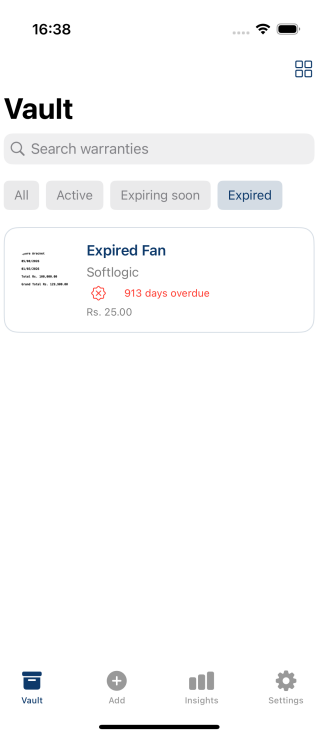


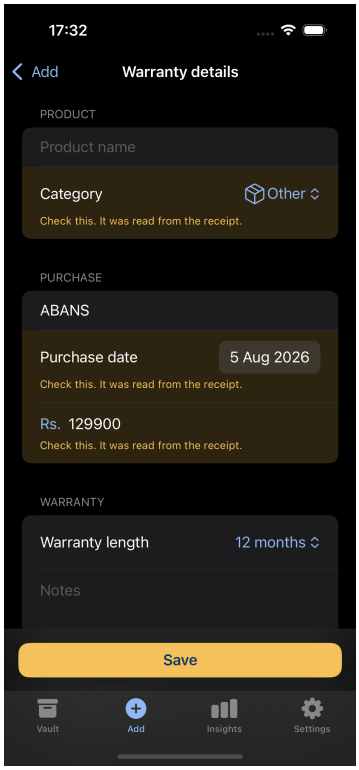


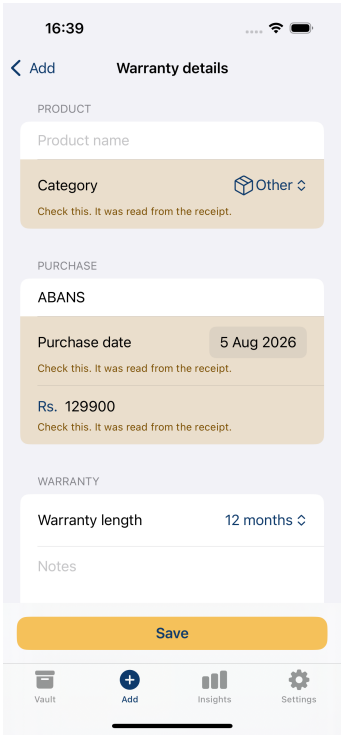


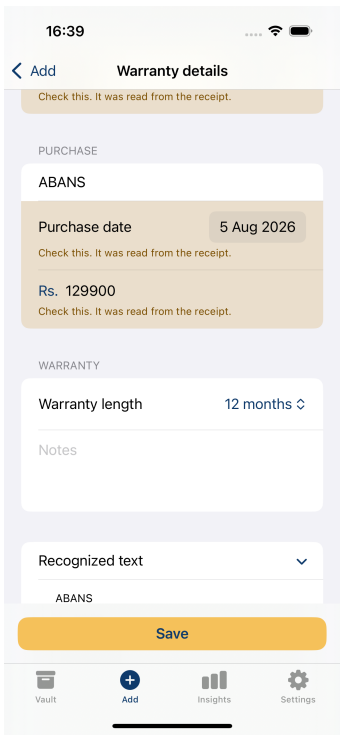


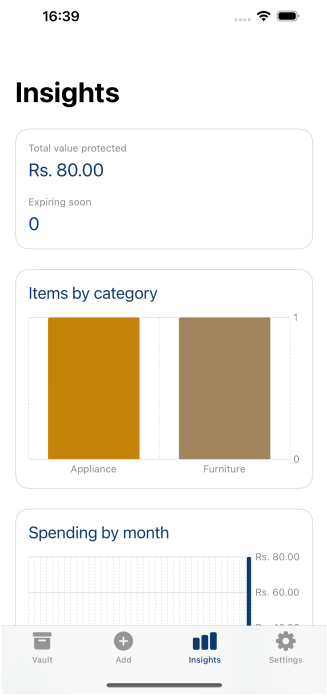




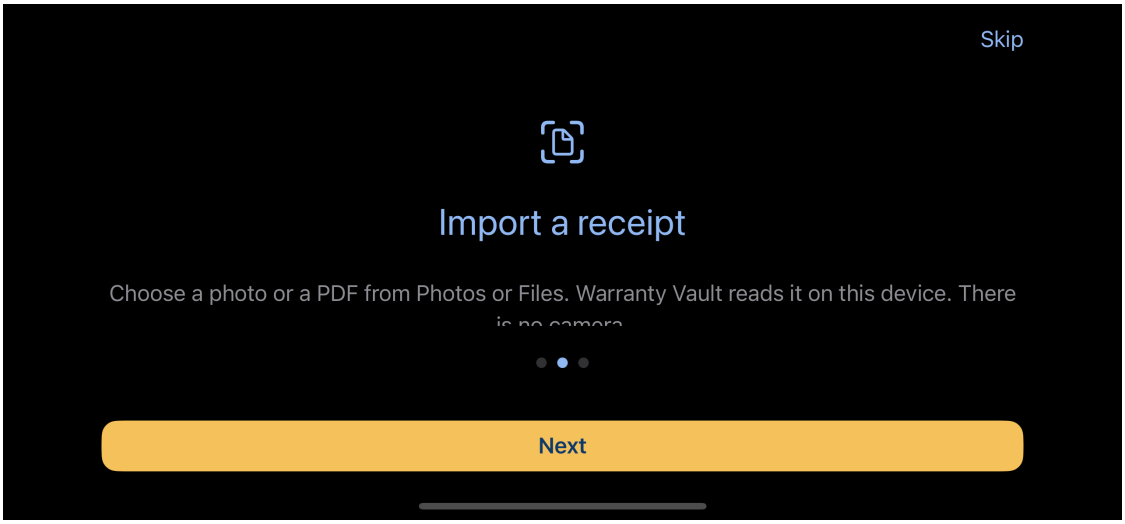


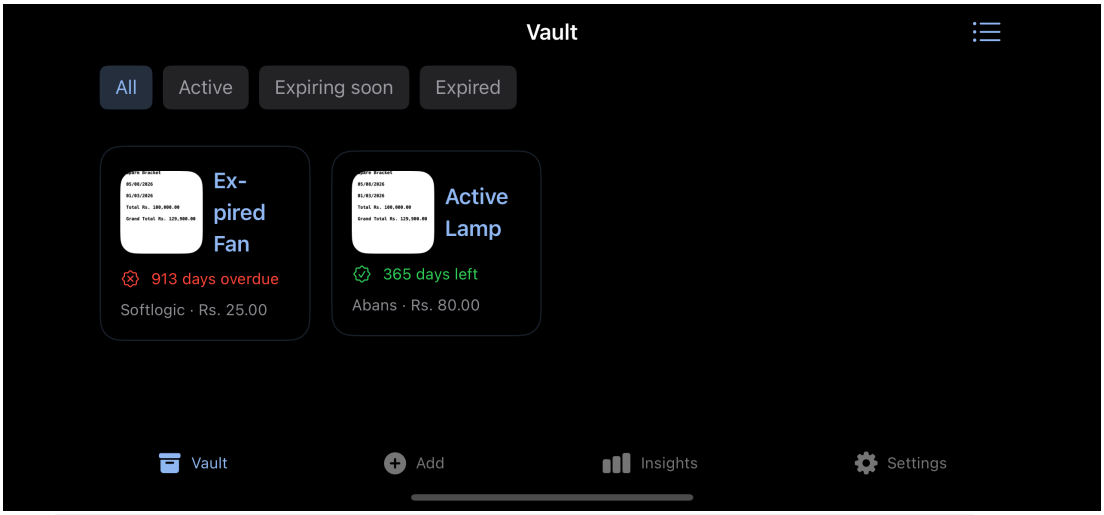


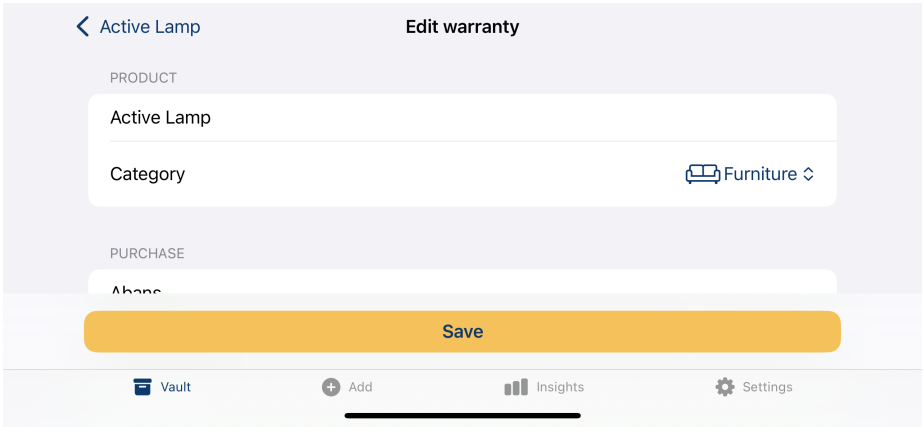




Screenshots of both light and dark modes included here in mix for comparison





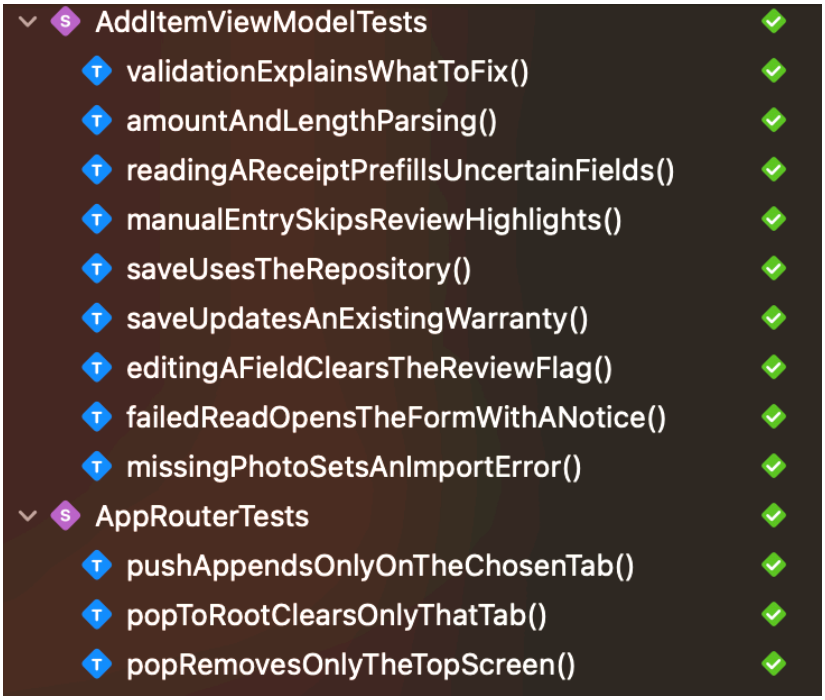


landscapes in both modes

7. Testing and results

The project includes unit tests for ReceiptParser, CategorySuggester, repository operations, reminders through a mock scheduler, AddItemViewModel, CSV export, summary calculations, and currency formatting. UI tests cover onboarding, manual item creation, filtering, and deletion. Launch arguments support repeatable test setup.

Test area	Results
Unit and UI tests	All tests passing
Synthetic receipt OCR	10 receipts tested
Store field	9/10 correct
Purchase date	8/10 correct
Total amount	9/10 correct
Category suggestion	8/10 correct
User acceptance	3 participants; average ease 4.3/5
Notification check	Test banner within 5 seconds; pending list shows 30/7/1-day reminders



8. Challenges and solutions

- A repository test crashed because the ModelContainer was released too early. The test setup was corrected to keep the container alive.
- A SwiftData query using partial propertiesToFetch crashed with an enum field. The partial fetch was removed, and status, search, and sorting remain in memory.

- Date extraction failed on a real receipt, so the date parser was expanded to handle labelled dates and common OCR errors.
- The parser selected a change value instead of the receipt total. Ranked total labels and exclusions for cash/change/tax-related lines were added.
- Xcode/iOS version mismatch, a missing simulator runtime, and low disk space interrupted development. The environment was adjusted and a large system cache was cleared.
- OCR varies with receipt quality. Confidence indicators and the raw-text review section help users identify uncertain results.

9. Limitations and future work

- Live camera capture is not included; the current flow imports images and PDFs.
- The app has been tested on iPhone 16 simulators running iOS 18.1 and 18.6, with a minimum target of iOS 17.6.
- Sinhala-language receipts have not been tested, and OCR quality depends on receipt condition.
- The actual 30/7/1-day reminders cannot be observed live during a short test; pending reminders and mock-scheduler tests are used for verification.
- TestFlight distribution is not available in the current setup because there is no paid developer account.
- Future improvements could include physical-device validation, live camera capture, more receipt layouts and languages, and expanded accessibility testing.

10. References and third-party resources

- Apple Developer Documentation — SwiftUI.
- Apple Developer Documentation — SwiftData.
- Apple Developer Documentation — Vision and VNRecognizeTextRequest.
- Apple Developer Documentation — UserNotifications.
- Apple Developer Documentation — PhotosPicker and fileImporter.
- Apple Developer Documentation — Swift Charts and String Catalogs.

Project repository: https://github.com/IT22079640/warranty-vault